



Plain Text Files

Part 2: Reading, Writing, and Appending to Files

Files can be accessed and altered in Python with reading, writing, and appending.

Reading, Writing, and Appending

Python can open text files in different modes depending on what you want to do.

"r" (read mode) reads an existing file without changing it.

"w" (write mode) writes to a file and replaces any existing contents.

"a" (append mode) adds new content to the end of an existing file.

Opening a File to Read

A file can be opened in **read mode** using **"r"**.

The **with** statement automatically closes the file when the indented block finishes.

The file must already exist or Python will produce an error.

```
with open("students.txt", "r") as file:  
    pass
```



This string value here needs to be the path to the file. Here we assume an existing file called **"students.txt"** that is in the same folder as our Python script.

Reading the Whole File as One String

The `read()` method reads the entire contents of a file as one string.

Any line breaks in the file are included in that string.

This is useful when you want to work with the whole file at once.

```
with open("students.txt", "r") as file:  
    text = file.read()  
  
print(text)
```

OUTPUT

```
Alex  
Sam  
Jordan
```

Reading the File Line by Line

The `readlines()` method reads the file and stores each line as an item in a list.

This makes it easier to process each line separately.

The newline character `\n` is usually kept at the end of each line.

```
with open("students.txt", "r") as file:
    lines = file.readlines()

print(lines)
```

OUTPUT

```
['Alex\n', 'Sam\n', 'Jordan\n']
```

Processing a List of Lines

Once file lines are stored in a list, they can be processed one at a time using a loop.

Each line can be cleaned, changed, or checked individually.

strip() is often used to remove the newline character.

```
with open("students.txt", "r") as file:  
    lines = file.readlines()  
  
for line in lines:  
    name = line.strip()  
    print(name.upper())
```

OUTPUT

```
ALEX  
SAM  
JORDAN
```

Opening / Creating a File to Write

A file can be opened in **write mode** using **"w"**.

If the file does not exist, Python creates it.

If it already exists, its current contents are removed.

```
with open("students.txt", "w") as file:  
    pass
```



This string value here needs to be the path to the file you are writing. Here we assume there is no file called **"students.txt"** because we will create it ourselves.

Writing a New File

The `write()` method adds text to a file that has been opened for writing.

If `students.txt` does not already exist, Python creates it.

Newline characters can be added so each value appears on its own line.

```
with open("students.txt", "w") as file:  
    file.write("Alex\n")  
    file.write("Sam\n")  
    file.write("Jordan\n")
```

Overwriting a File

Opening an existing file with **"w"** completely replaces its current contents.

This means anything already stored in **students.txt** will be erased before new text is written.

After the operation, only the new data remains.

```
with open("students.txt", "w") as file:  
    file.write("Taylor\n")
```

Writing a File with Multiple Lines

A list of strings can be written to the file using a loop.

Each value can be followed by `\n` so it appears on its own line.

This is useful when the data is already stored in a Python list.

```
students = ["Alex", "Sam", "Jordan"]  
  
with open("students.txt", "w") as file:  
    for student in students:  
        file.write(student + "\n")
```

Opening / Creating a File to Append

A file can be opened in **append mode** using **"a"**.

If the file does not exist, Python creates it.

If it does exist, the current contents are kept.

```
with open("students.txt", "a") as file:  
    pass
```

This string value here needs to be the path to the file you are appending.

"students.txt" doesn't need to exist, but it can.

Adding to an Existing File

Append mode adds new text to the end of the file instead of replacing what is already there.

This makes it useful when new records need to be added over time.

A newline can be included so the new value appears on a separate line.

```
with open("students.txt", "a") as file:  
    file.write("Taylor\n")
```

Appending Multiple Lines to a File

Multiple values can be added to the end of a file by looping through a list.

The existing contents remain unchanged.

Each new value is written after the previous data.

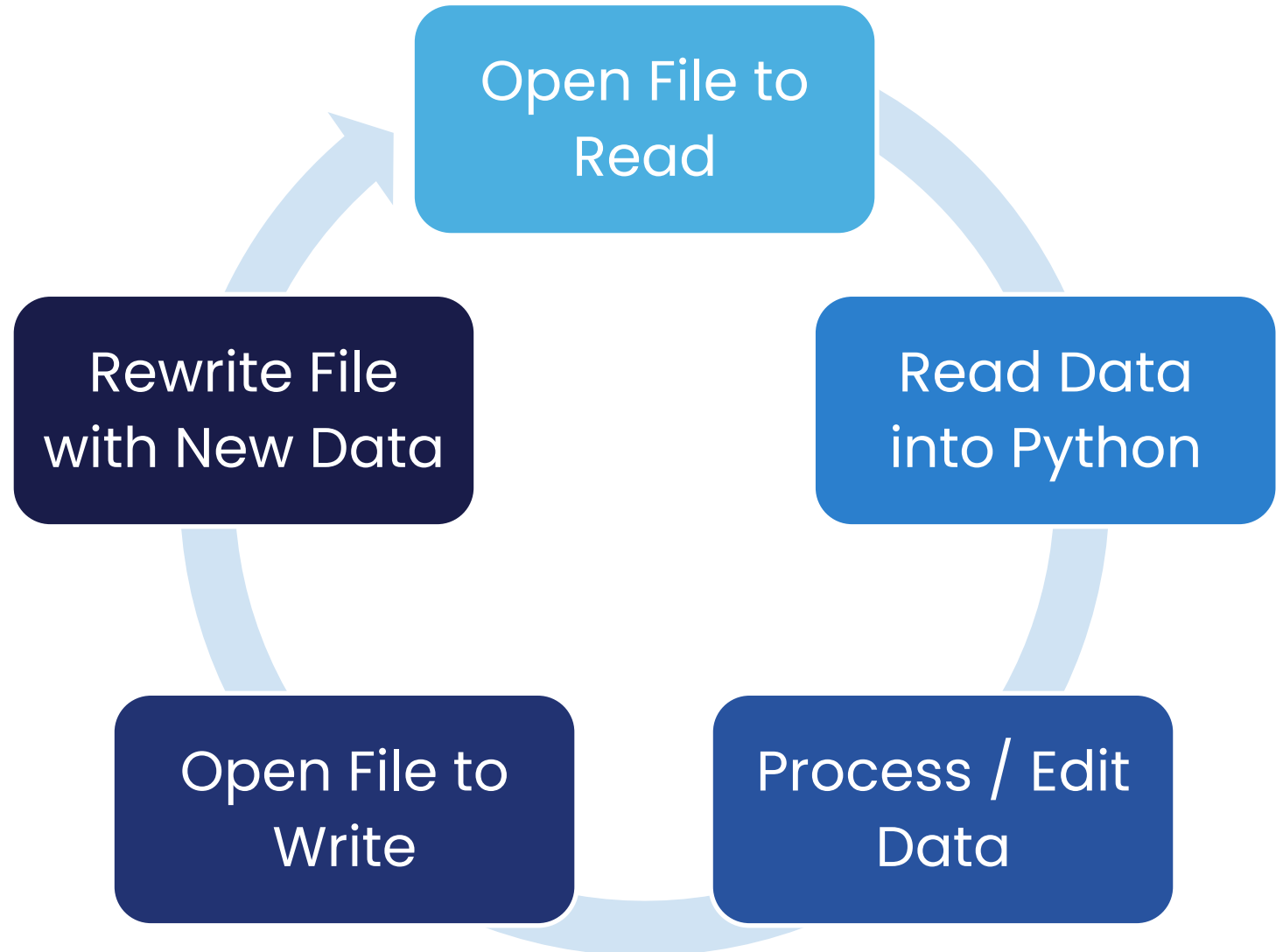
```
new_students = ["Taylor", "Morgan"]  
  
with open("students.txt", "a") as file:  
    for student in new_students:  
        file.write(student + "\n")
```

Editing a File with Read and Write

Text files are usually edited by first in **"r"** mode by reading the current data into Python.

The program then modifies that data in memory and rewrites the file using **"w"** mode.

This creates a simple read-process-write cycle.



Code Example of Reading, Processing, and Writing a File

```
with open("students.txt", "r") as file:
    students = file.readlines()

processed_students = []

for student in students:
    student = student.strip().title()
    processed_students.append(student)

with open("students.txt", "w") as file:
    for student in processed_students:
        file.write(student + "\n")
```